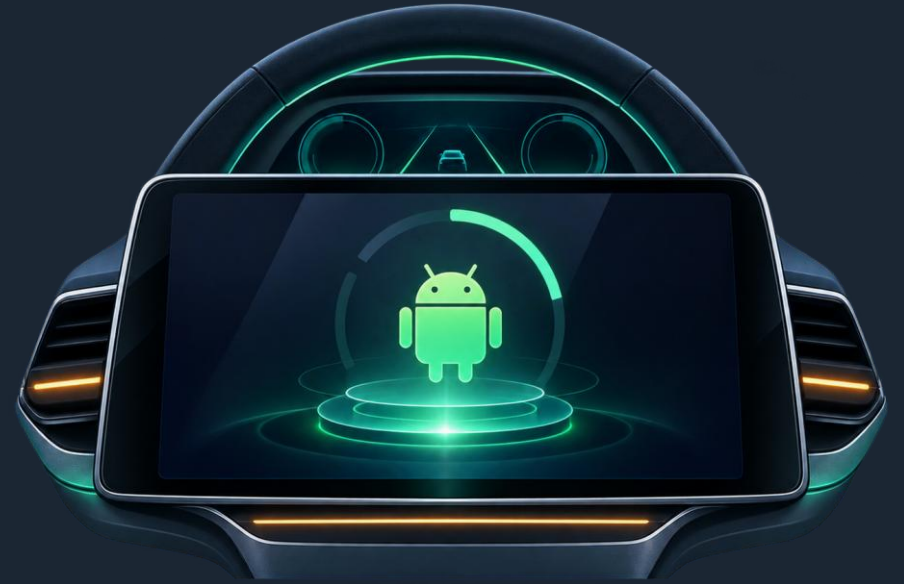


Android Framework 16 심화

Day 1 — 환경 설정 · Architecture · Boot · Binder / AIDL



오늘의 일정 — 8시간

Part	내용	소요
Part 0 환경 설정 실습	실습 1 빌드 서버 접속 · 실습 2 Android Studio/Emulator · 실습 3 adb · 실습 4 AOSP 빌드	100분
Part 1 강의	Ch 0 Architecture & Boot · Ch 1 Binder · Ch 2 AIDL · Ch 3 Framework 내장 IPC	140분
Part 2 실습	실습 5 ~ 11 (시스템 탐험, Binder 탐정, AIDL 계산기, DeathRecipient, oneway+Callback, Messenger, TransactionTooLarge) · 실습 12 선택	205분
Part 3 데모 · 정리	실습 13 Kernel Module 데모 · Worksheet	35분



오전 목표: 실습 4까지 전원 완료 — 커스텀 Emulator 부팅 + my_sp 실행 확인. 이것이 오후 실습과 Day 2~4 전체의 전제 조건입니다.

Part 0 · 환경 설정 실습 (실습 1 ~ 4)

1

빌드 서버 접속

PuTTY · WinSCP 설치, Ed25519 키, ID 없이 자동 로그인

35분

2

Android Studio · Emulator

Automotive AVD (API 35 Google APIs), PATH 등록

20분

3

adb 접속 방법

adb root / shell / push / remount / logcat / dumpsys

15분

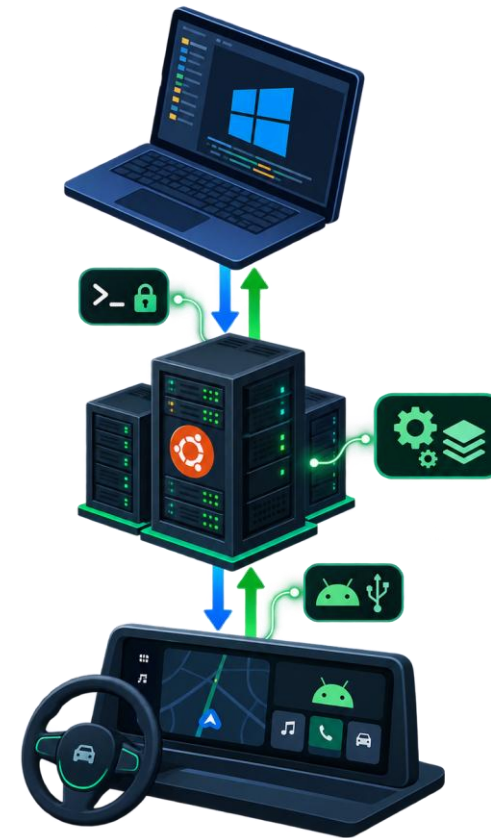
4

빌드 서버에서 Android 빌드

lunch → m → emu_img_zip → 커스텀 Emulator → mm → 배포

30분

▶ 애니메이션 8 · AOSP 빌드·배포 파이프라인



Chapter 0

Android Architecture & Boot

50분 · Linux 위의 Android · init → Zygote → system_server · ServiceManager



Android Architecture — 5개 계층

Application

Car Launcher · Settings · 실습 App (BinderLab, StockLab ...)

Framework (Java)

ActivityManager · WindowManager · PackageManager · CarService
— system_server 안에서 실행

Native / ART

libbinder · SurfaceFlinger · servicemanager · ART 런타임 · Bionic libc

HAL

Vehicle HAL · Audio HAL · Graphics HAL — AIDL/HIDL 인터페이스,
vendor 파티션

Linux Kernel (GKI 6.12)

Binder Driver · ashmem/dmabuf · SELinux · LMKD 지원 · 커널 모듈(.ko)

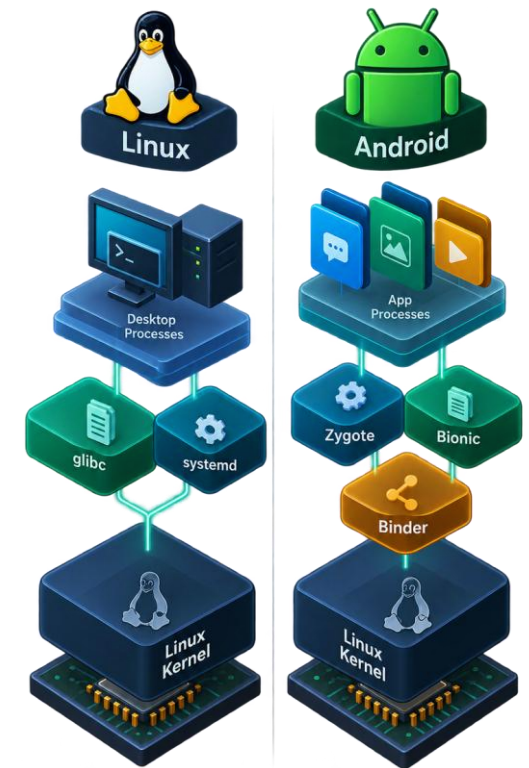
이 과정에서 다루는 범위

- Day 1: Framework ↔ Native 경계 — Binder, AIDL
- Day 2: Native Binder Service, HAL, SELinux
- Day 3: JNI, SurfaceFlinger, Memory/LMKD
- Day 4: APEX, RRO, SystemUI, Automotive (CarService, V HAL)
- App 계층은 실습용 Client로만 사용

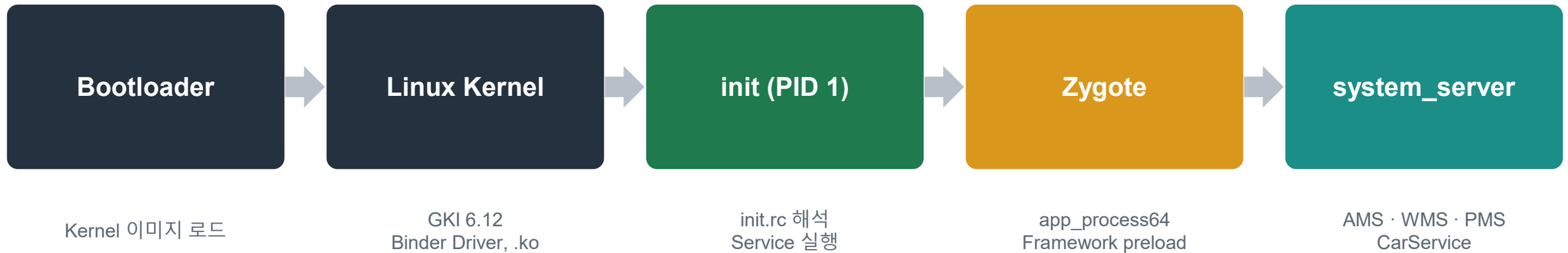


Linux vs Android — 무엇이 다른가

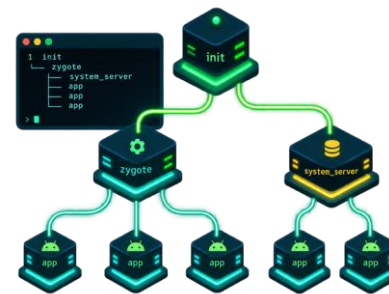
항목	일반 Linux	Android
libc	glibc	Bionic (경량, BSD 라이선스)
init	systemd / SysV init	Android init + init.rc (service, on trigger)
IPC	Socket, Pipe, D-Bus, Shared Memory	Binder (Driver 기반, 1-copy, 참조 카운팅, UID 전달)
Process 생성	fork + exec	Zygote fork (preload 공유, COW)
메모리 부족	kernel OOM killer	LMKD + oom_adj 점수 (App 우선순위 기반)
공유 메모리	shm, mmap	ashmem / dmabuf (FD 로 Binder 전달)
보안	DAC + SELinux (선택)	DAC + SELinux 필수 (enforcing) + App 별 UID
화면	X11 / Wayland	SurfaceFlinger + HWC



Boot Sequence — Bootloader에서 system_server까지



```
# init.zygote64_32.rc
service zygote /system/bin/app_process64 -Xzygote /system/bin \
  --zygote --start-system-server --socket-name=zygote
class main
priority -20
user root
socket zygote stream 660 root system
```



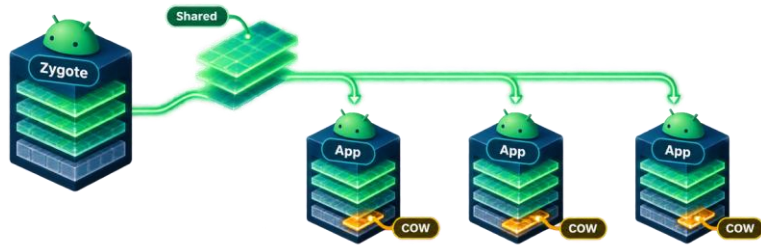
- --start-system-server → Zygote가 system_server를 fork
- socket zygote — AMS가 App 실행을 요청하는 통로
- 실습 5: init.zygote64_32.rc 확인

▶ [애니메이션 1 · Boot Sequence](#)

Zygote와 system_server — 왜 fork인가

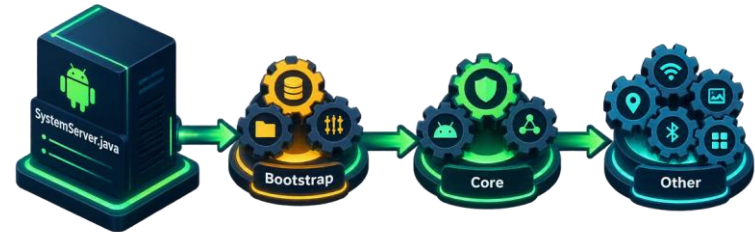
Zygote

- Framework 클래스 · 리소스 · ART 를 한 번만 preload
- AMS 가 socket 으로 요청 → fork() → App Process
- Copy-on-Write: preload 된 페이지를 모든 App 이 공유
- App 시작이 빠르고 메모리 절약
- 실습 5 확인: 모든 App 의 PPID = zygote64 PID

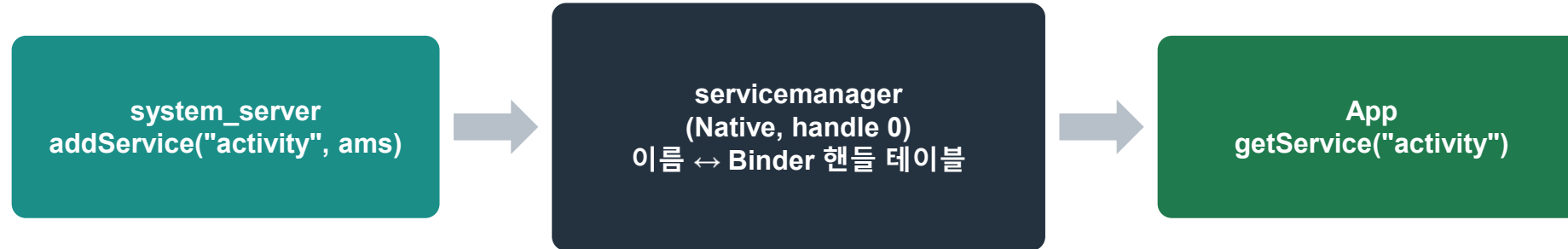


system_server

- Zygote 가 --start-system-server 로 fork 한 첫 자식
- SystemServer.java: startBootstrapServices → startCoreServices → startOtherServices
- AMS · WMS · PMS · PowerManager · CarService ...
- 각 Service 는 ServiceManager 에 이름으로 등록 (service list)
- App ↔ system_server 통신은 예외 없이 Binder

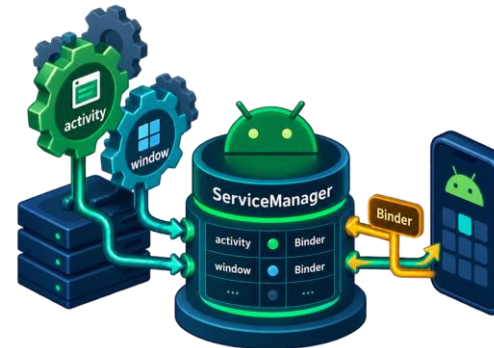


ServiceManager — Binder Service의 전화번호부



```

$ adb shell service list
Found 212 services:
0  sip: [android.net.sip.ISipService]
1  phone: [com.android.internal.telephony.ITelephony]
2  activity: [android.app.IActivityManager]
3  window: [android.view.IWindowManager]
...
$ adb shell service call clipboard 1 # code 1
  
```



- servicemanager = Binder handle 0
- 이름 → IBinder → asInterface() → Proxy
- service call: Proxy 없이 Driver 직접 (실습 6)
- Day 2: led_auth_service 를 addService

시스템 탐험가 — adb로 Android 내부 들여다보기

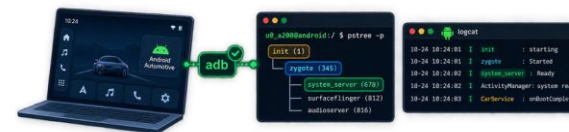
목표 Process 트리 · System Service 목록 · Binder 상태를 눈으로 확인 · 소요 20분

진행 순서

1. adb root → adb shell → ps -A -o pid,ppid,name | head -30
2. ps -A | grep zygote / grep system_server — PPID 관계 확인
3. service list | wc -l, dumpsys activity | head
4. cat /dev/binderfs/binder_logs/stats, proc/<system_server PID>
5. cat /system/etc/init/hw/init.zygote64_32.rc — --start-system-server 찾기

확인 포인트

- Zygote PPID = 1 (init)
- system_server PPID = zygote64 PID
- 모든 App PPID = zygote64
- 보고서: PID 기록, Service 개수, Transaction 상위 3개

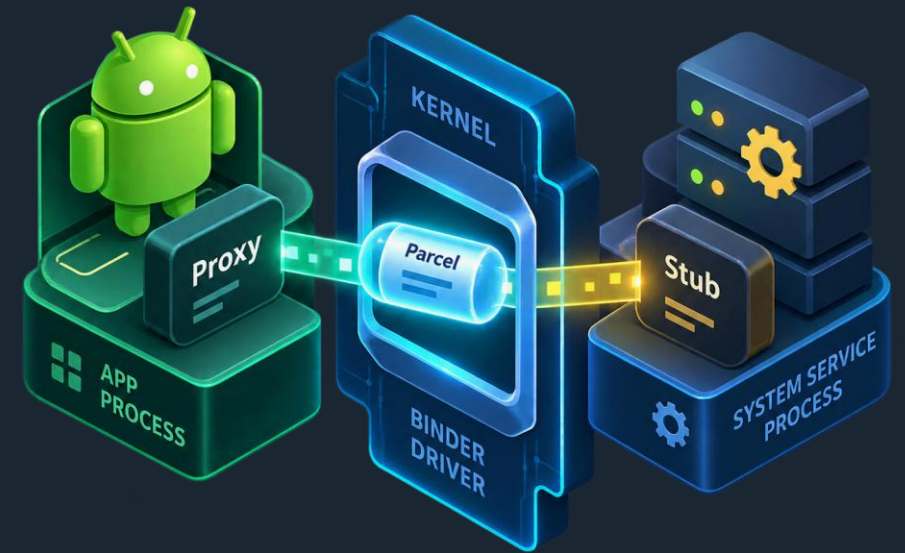


▶ [애니메이션 1 · Boot Sequence 다시 보기](#)

Chapter 1

Binder — Android의 표준 IPC

45분 · Driver · Proxy/Stub · Thread Pool · 보안 · Transaction 한계



왜 Socket이 아니라 Binder인가

1 1-copy 전달

송신 Process → 수신 Process 의 mmap 된 Buffer 로 커널이 한 번만 복사. Socket 은 user→kernel→user 2회.

2 호출자 신원 보증

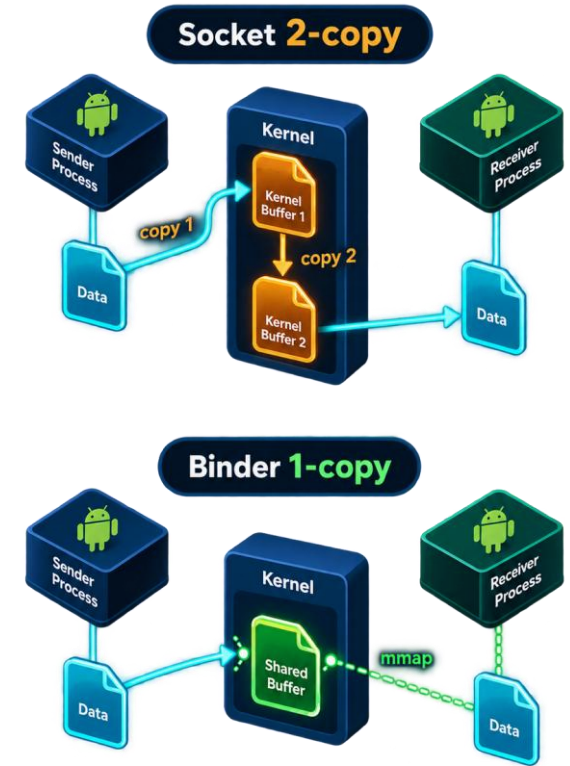
Driver 가 getCallingUid()/Pid() 를 전달 — Socket 은 상대가 누구인지 믿을 수 없다. Android 권한 검사 기반.

3 객체 참조 · 사망 통지

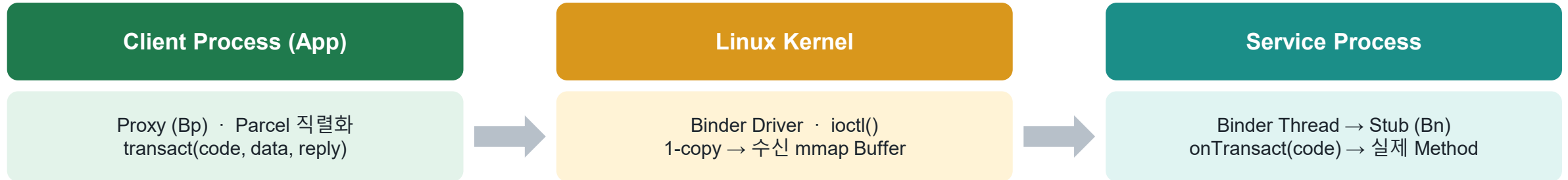
IBinder 는 참조 카운팅되는 객체. 상대 Process 가 죽으면 DeathRecipient 로 즉시 통지.

4 Thread 관리

Driver 가 수신 Process 의 Binder Thread Pool 을 직접 깨운다. 개발자가 accept 루프를 쓰지 않는다.

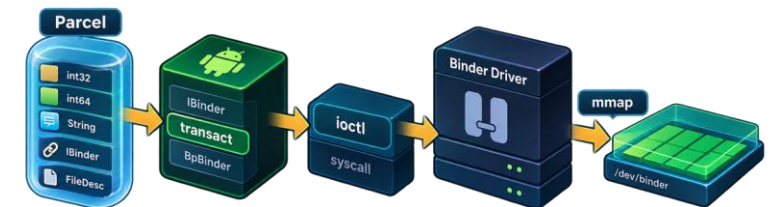


Binder IPC 흐름 — Proxy → Driver → Stub



reply Parcel 은 같은 경로로 되돌아온다 — 동기 호출이면 Client Thread 는 그때까지 블로킹

단계	Client 코드에서 보이는 것	실제로 일어나는 일
1	service.add(42, 7)	Proxy.add() 진입 — 그냥 Method 호출처럼 보인다
2	(없음)	인자를 Parcel 에 쓰고 TRANSACTION_add(=1) 과 함께 mRemote.transact()
3	(없음)	ioctl(BINDER_WRITE_READ) → Driver 가 수신 Process Buffer 로 1-copy
4	(없음)	Driver 가 Service 의 Binder Thread 를 깨움 → Stub.onTransact(code=1)
5	(없음)	CalculatorService.add() 실행 → reply Parcel 작성 → 역방향
6	int r = 49	Proxy 가 reply 에서 int 를 읽어 반환 — 여기까지 Client Thread 블로킹



▶ [애니메이션 2 · Binder IPC 흐름](#)

▶ [애니메이션 5 · AIDL → Stub / Proxy](#)

Binder Thread Pool과 호출자 확인

Binder Thread Pool

- Service Process 마다 Driver 가 관리하는 Thread 풀 (기본 최대 16)
- 요청이 동시에 오면 각각 다른 Binder Thread 에서 병렬 실행
- → Stub Method 는 Thread-safe 하게: synchronized / ConcurrentHashMap
- 콜백 · DeathRecipient 도 Binder Thread 에서 호출
- → UI 접근은 runOnUiThread() / Handler.post()
- Messenger 는 Handler 큐로 단일 Thread 순차 처리 (Ch 3)

호출자 확인 · 권한 검사

```
int uid = Binder.getCallingUid(); // 커널이 보증
int pid = Binder.getCallingPid();
if (uid != Process.SYSTEM_UID)
    throw new SecurityException();

long token = Binder.clearCallingIdentity();
try { /* 내부 호출은 Service 자신의 UID */ }
finally { Binder.restoreCallingIdentity(token); }
```



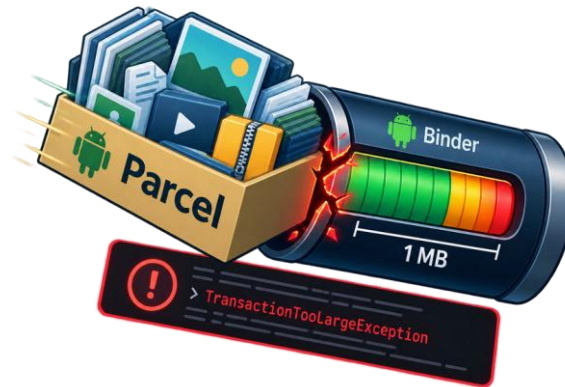
Transaction Buffer 한계 — TransactionTooLargeException

다른 Transaction 점유

+640 KB 요청 → 초과

Process 당 1 MB

- Buffer 는 Process 당 1 MB, 모든 진행 중 Transaction 이 공유
→ 실질 한계는 그 절반
- byte[] 를 reply 에 통째로 쓰면 Driver 가 거부
- Intent extras · Bundle · savedInstanceState 도 같은 Buffer →
큰 Bitmap 은 크래시 원인



해결: FD 만 Parcel 에 실는다

- ParcelFileDescriptor.createPipe() → Parcel 에는 FD 4 byte
- Driver 가 FD 를 수신 Process 에 복제 → 데이터는 Pipe 로 스트리밍
- ashmem / SharedMemory / 파일도 같은 원리

```
ParcelFileDescriptor[] p = ParcelFileDescriptor.createPipe();
new Thread(() -> { write(p[1], data); p[1].close(); }).start();
return p[0]; // Parcel 에는 FD 하나만
// Client: new FileInputStream(pfd.getFileDescriptor()).read(buf)
```

▶ [애니메이션 7 · Transaction Buffer 한계](#)

Binder 탐정 — System Service 추적하기

목표 App 이 System Service 를 호출할 때 발생하는 Binder Transaction 을 실시간 관찰 · 소요 20분

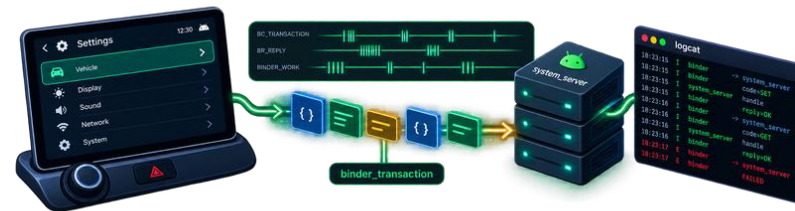
진행 순서

1. adb shell → service call clipboard 1 (Proxy 없이 Driver 직접 호출)
2. echo 1 > /sys/kernel/tracing/events/binder/binder_transaction/enable
3. echo 1 > tracing_on → cat trace_pipe
4. Emulator 에서 Settings 앱 열기 → Transaction 로그 관찰
5. SETPID=\$(pidof com.android.car.settings) → binder_logs/proc/\$SETPID

확인 포인트

- dest_proc 가 system_server PID 인 Transaction 이 대부분
- reply=0 요청 / reply=1 응답 쌍
- flags 0x1 = oneway
- 결과물: Settings 앱 열 때 Transaction 수, 호출된 Service 추정

▶ 애니메이션 2 · Binder IPC 흐름



TransactionTooLargeException 재현 & 해결

목표 Buffer 한계를 직접 재현하고 ParcelFileDescriptor(Pipe) 로 해결 · 소요 20분

진행 순서

1. 프로젝트 BinderLimitLab, IDataService.aidl (byte[] getDataDirect / ParcelFileDescriptor getDataViaPipe)
2. DataService: byte[] 반환 vs createPipe + PipeWriter Thread
3. android:process=":data_remote" 로 분리
4. 버튼 1: 10 KB → 2 MB 2배씩 증가 → 예외 발생 지점 기록
5. 버튼 2: Pipe 로 4 MB 수신 · 버튼 3: 256 KB 속도 비교

확인 포인트

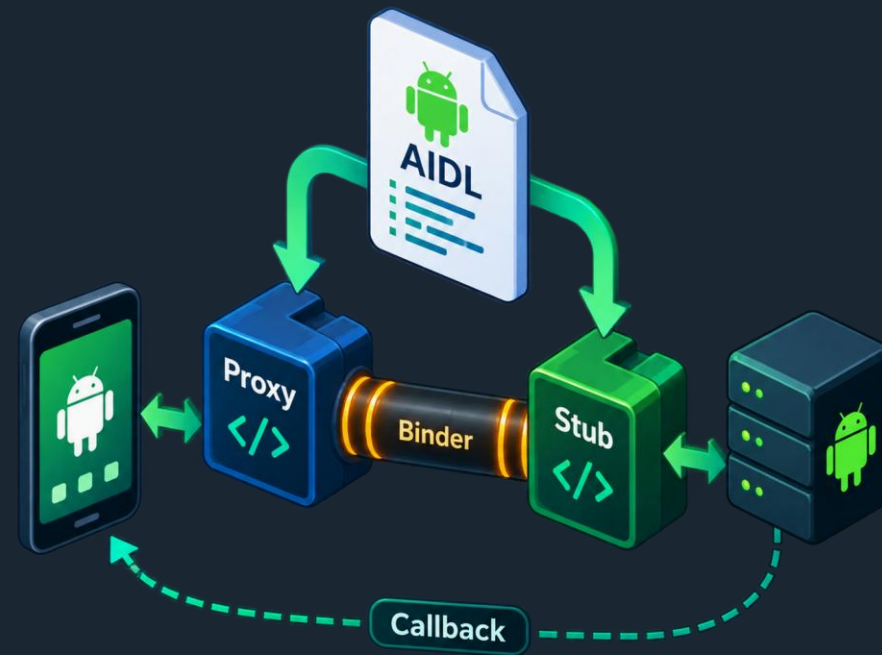
- 640 KB 또는 1 MB 근처에서 TransactionTooLargeException
- Pipe 4096 KB 정상 수신
- 소용량은 직접 전달이 약간 빠르거나 비슷
- 결론: 대용량은 항상 FD

▶ [애니메이션 7 · Transaction Buffer 한계](#)



AIDL & IPC 패턴

30분 · AIDL 문법 · 동기 vs oneway · Callback · DeathRecipient · android:process



AIDL 기본 — 인터페이스 한 파일에서 Stub / Proxy까지

```
// app/src/main/aidl/com/example/binderlab/ICalculatorService.aidl
package com.example.binderlab;

interface ICalculatorService {
    int add(int a, int b);
    double divide(int a, int b);
    String getCallerInfo();
    List<String> getHistory();
}
```

```
// build.gradle.kts
android { buildFeatures { aidl = true } }

// AndroidManifest.xml
<service android:name=".CalculatorService"
    android:exported="true"
    android:process=":calc_remote" />
```

요소	설명
지원 타입	기본형, String, CharSequence, List, Map, Parcelable, IBinder, 다른 AIDL 인터페이스
방향 태그	in (기본) / out / inout — 배열·Parcelable 에 필요
Stub	Server 가 상속. onTransact(code) → 실제 Method. Binder Thread 에서 실행
Proxy	Client 가 asInterface() 로 획득. Parcel 직렬화 + transact()
android:process	":name" 으로 Service 를 별도 Process 에 → 실제 IPC 발생
bindService	명시적 Intent (setPackage 필수) + BIND_AUTO_CREATE



▶ 애니메이션 5 · AIDL → Stub / Proxy

동기 vs oneway — 그리고 Callback

동기 `getPrice("AAPL")`

Client Main Thread 블로킹 2000 ms — UI 정지 → ANR 위험

`oneway subscribe("AAPL", callback)`

0 ms 즉시 반환 → UI 계속 동작 · 결과는 콜백(cb)으로

cb

cb

cb

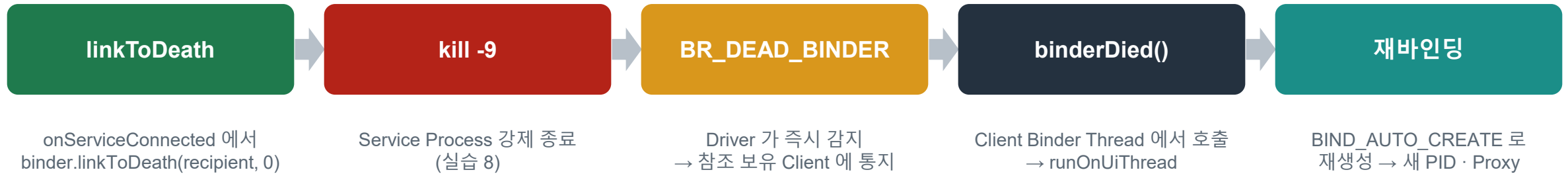
```
interface IStockCallback {
    oneway void onPriceUpdate(String symbol, double price, long ts);
}
interface IStockService {
    double getPrice(String symbol); // 동기
    oneway void subscribe(String symbol, IStockCallback cb); // 비동기
}
```



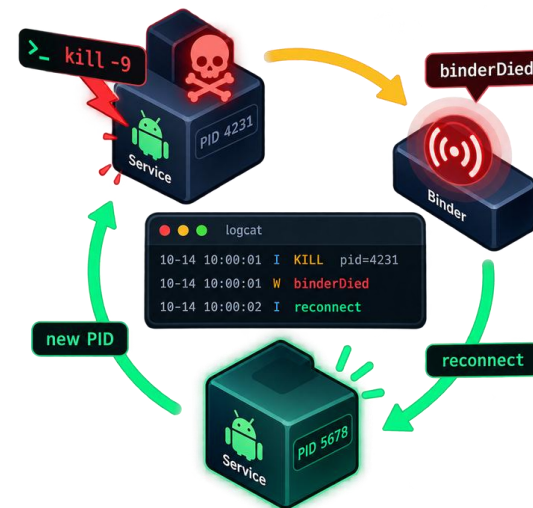
- oneway: reply 없음, 반환값 불가, 즉시 반환
- Callback = Client 가 넘기는 Binder 객체 (역방향 IPC)
- RemoteCallbackList: Client 사망 시 자동 제거
- 콜백은 Binder Thread → `runOnUiThread()`
- Main Thread 동기 IPC 는 ANR 의 원인

▶ [애니메이션 3 · 동기 vs oneway + Callback](#)

DeathRecipient — Service 사망 감지와 자동 복구



```
private final IBinder.DeathRecipient deathRecipient = () -> {
    Log.e(TAG, "died on " + Thread.currentThread().getName()); // Binder:2345_2
    service = null;
    runOnUiThread(() -> infoText.setText("☠ Service DIED! Rebinding.."));
    unbindService(connection);
    bindService(intent, connection, Context.BIND_AUTO_CREATE);
};
// onServiceConnected: binder.linkToDeath(deathRecipient, 0);
```



Logcat 순서	Thread
1 binderDied()	Binder Thread
2 onServiceDisconnected()	Main (AMS 경유)
3 onServiceConnected()	Main, 새 PID

- 새 Process → 내부 상태 초기화
- 옛 Proxy → DeadObjectException

▶ 애니메이션 4 · DeathRecipient

AIDL 계산기 — 기본 IPC Service 구현

목표 AIDL 정의 → Service(Stub) → Client(Proxy) 구현, 별도 Process 에서 실제 IPC 발생 · 소요 45분

진행 순서

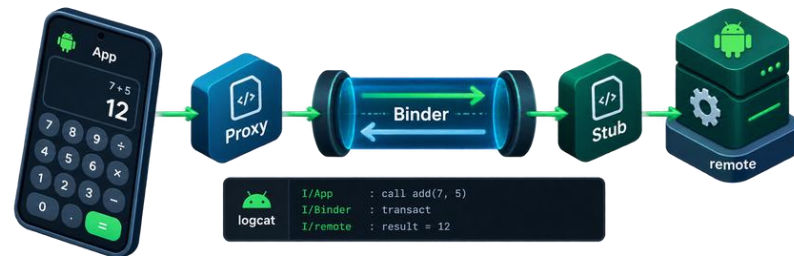
1. 프로젝트 BinderLab (Java, API 36) · buildFeatures { aidl = true }
2. ICalculatorService.aidl 작성 → Make Project → Stub 자동완성 확인
3. CalculatorService: Stub 구현, getCallerInfo(), clearCallingIdentity 패턴
4. Manifest: android:process=":calc_remote"
5. MainActivity: bindService(setPackage 필수) → Stub.asInterface()
6. 실행 후 ps | grep binderlab 로 2개 Process 확인

확인 포인트

- 화면 상단 "Same process: NO"
- android:process 제거 후 재실행 → "Same process: YES"
- Logcat: Method 가 Binder:xxxx_N Thread 에서 실행
- divide(1, 0) → 예외가 Client 로 전달

▶ 애니메이션 5 · AIDL → Stub / Proxy

▶ 애니메이션 2 · Binder IPC 흐름



Binder 생존 게임 — DeathRecipient & 복구

목표 실습 7 프로젝트에 DeathRecipient 를 추가하고 kill -9 로 자동 복구를 체험 · 소요 30분

진행 순서

1. MainActivity 에 IBinder.DeathRecipient 필드 추가 (binderDied → runOnUiThread → 재바인딩)
2. onServiceConnected: binder.linkToDeath(deathRecipient, 0)
3. onDestroy: unlinkToDeath
4. adb shell kill -9 \$(pidof com.example.binderlab:calc_remote)
5. for 루프로 3초마다 5회 반복 kill

확인 포인트

- 화면: "☹️ Service DIED! Rebinding... (#N)"
- Logcat 순서: binderDied → onServiceDisconnected → onServiceConnected
- binderDied 의 Thread = Binder:xxxx_N
- Service PID 변경, 계산 다시 동작

▶ 애니메이션 4 · DeathRecipient & 복구



비동기 주식 시세 Service — oneway + Callback

목표 동기 2초 블로킹 vs oneway 0 ms 를 체험하고 Callback · RemoteCallbackList 구현 · 소요 40분

진행 순서

1. 프로젝트 StockLab · IStockCallback.aidl (oneway) + IStockService.aidl
2. StockService: getPrice 2초 sleep, subscribe → RemoteCallbackList.register, 1초 주기 broadcast
3. MainActivity: IStockCallback.Stub 구현, 콜백 안 runOnUiThread
4. android:process=":stock_remote"
5. 동기 버튼 → UI 정지 관찰 / 비동기 버튼 → 즉시 반환 + 1초마다 시세

확인 포인트

- 동기: 소요 2000 ms, 화면 멈춤
- oneway: 반환 0 ms
- 콜백 로그 thread=Binder:xxxx_N
- kill -9 stock_remote → binderDied 가 먼저

▶ 애니메이션 3 · 동기 vs oneway + Callback

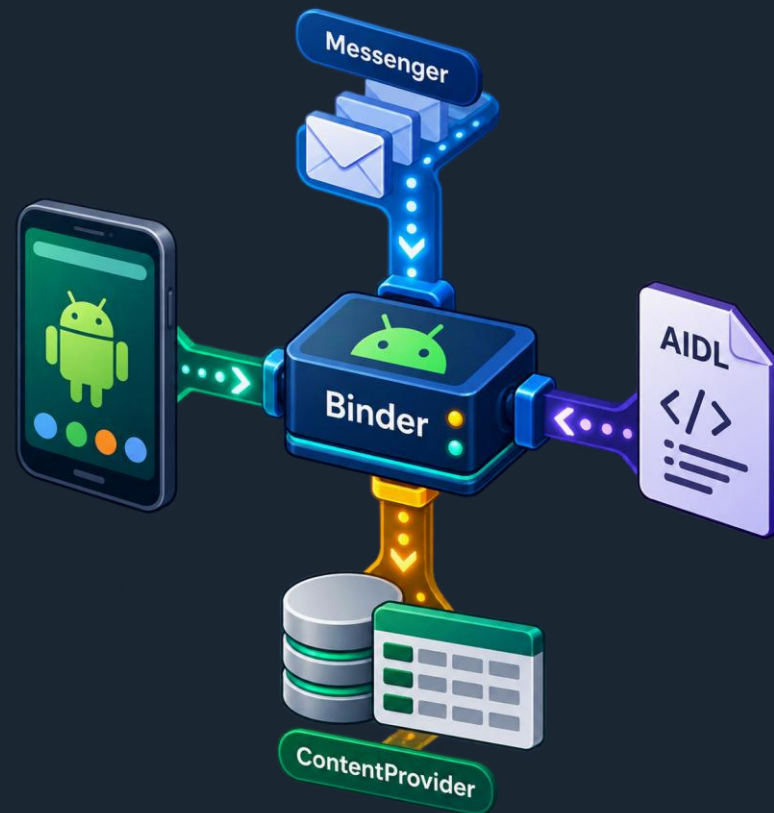
▶ 애니메이션 6 · Binder Thread Pool



Chapter 3

Framework 내장 IPC — Messenger · Content Provider

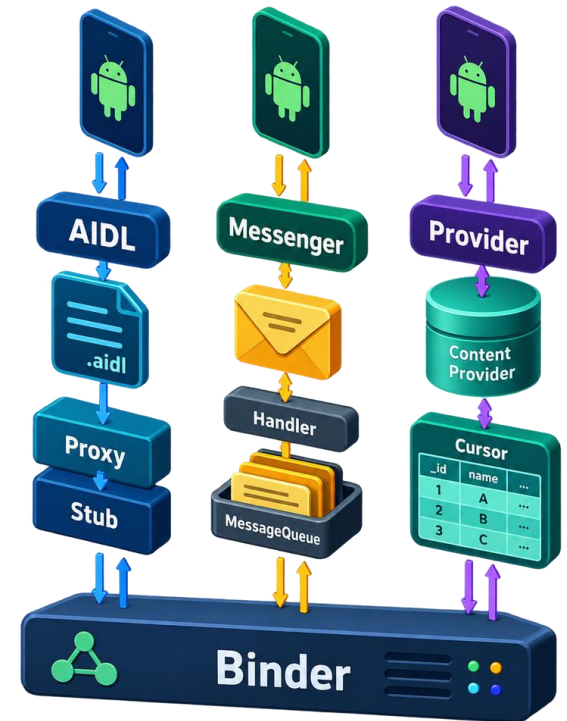
15분 · AIDL 없이 Binder 를 쓰는 두 가지 방법 · 언제 무엇을 쓸 것인가



Messenger vs Content Provider vs AIDL

	AIDL	Messenger	Content Provider
.aidl 파일	필요	불 필요	불 필요
요청 구분	타입 안전한 Method	Message.what 정수 + Bundle	URI + CRUD (query/insert/update/delete)
Thread 모델	Binder Thread 병렬 → lock 필요	Handler 큐 단일 Thread 순차 → lock 불 필요	Binder Thread 병렬 (Provider 구현이 처리)
응답	동기 반환값 / oneway + Callback	replyTo Messenger 로 비동기	Cursor (CursorWindow = ashmem FD)
변경 알림	Callback 직접 구현	브로드캐스트 직접 구현	ContentObserver · notifyChange()
외부 접근	bindService	bindService	adb shell content query/insert
적합한 곳	성능·타입 중요, 복잡한 API	간단한 명령·알림, 순서 보장	구조화 데이터 공유, 다른 App 에 개방

셋 다 같은 Binder Driver 위에서 동작한다. Message 도 Cursor 도 결국 Parcel 로 직렬화되어 transact() 를 탄다.



Messenger 채팅 앱 — 경량 IPC

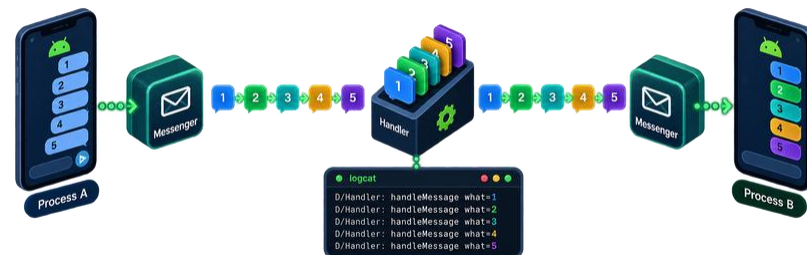
목표 AIDL 없이 Messenger 로 양방향 통신, Handler 단일 Thread 순차 처리 확인 · 소요 30분

진행 순서

1. 프로젝트 MessengerChat (aidl 설정 불필요)
2. ChatConstants: MSG_REGISTER / MSG_SEND_TEXT / MSG_GET_TIME ... 정수 상수
3. ChatService: IncomingHandler(Main Looper) → new Messenger(handler).getBinder()
4. MainActivity: new Messenger(binder) + replyTo = clientMessenger
5. android:process=":chat_remote" · 테스트 5가지 (Echo, 시간, 연속 5개, ps, kill)

확인 포인트

- 전송 후 Echo + 브로드캐스트 2줄 수신
- 연속 5개가 전송 순서 그대로 처리
- ps: 앱 / :chat_remote 2개 Process
- kill 후 BIND_AUTO_CREATE 로 자동 재시작



Content Provider 메모장 — 데이터 공유 (선택 / 과제)

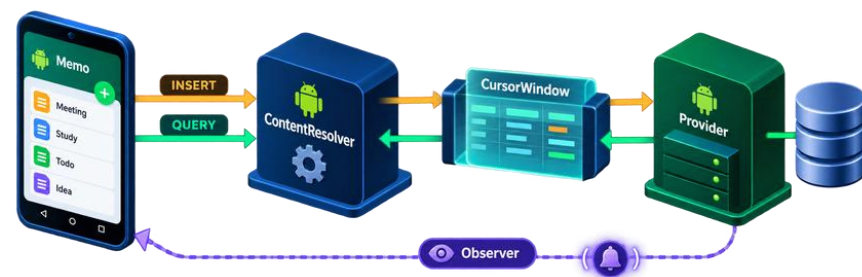
목표 ContentResolver 내부의 Binder 와 CursorWindow(ashmem FD), ContentObserver 를 확인 · 소요 45분

진행 순서

1. 프로젝트 MemoProvider · MemoContract (AUTHORITY, URI, 컬럼)
2. MemoDbHelper (SQLite) + MemoProvider (UriMatcher, CRUD, notifyChange)
3. Manifest <provider android:process=":provider_remote">
4. MainActivity: ContentResolver insert/query/delete + ContentObserver
5. adb shell content insert/query/delete 로 외부 접근

확인 포인트

- Provider 로그의 caller uid/pid 가 Activity 와 다름
- 큰 결과도 CursorWindow 로 전달 (Buffer 한계 회피)
- adb 로 insert 하면 앱 화면 Observer 즉시 반응



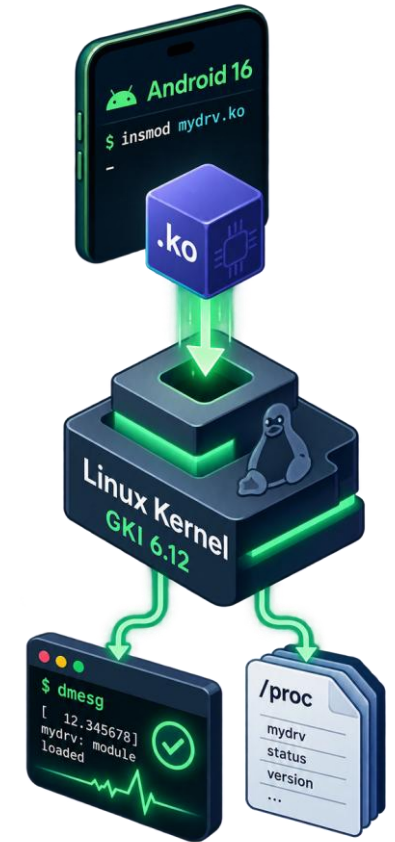
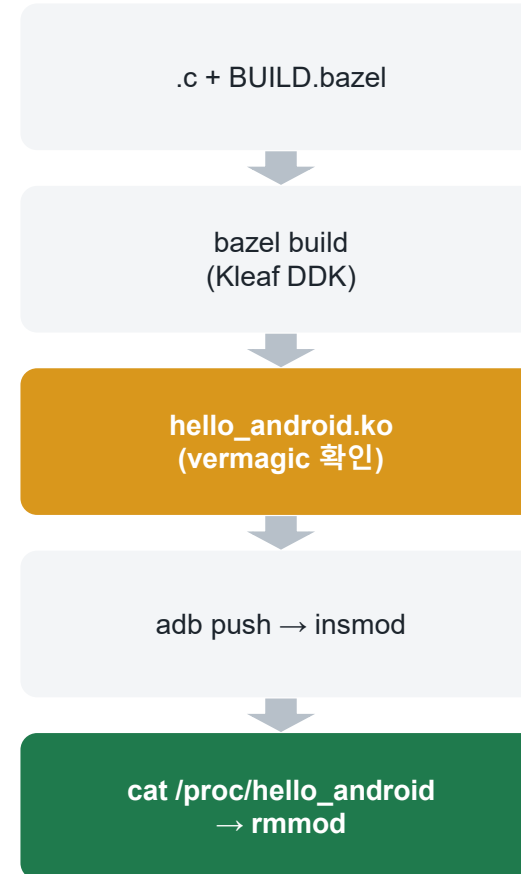
실습 13 (데모) · Android 16 Kernel Module — ddk_module

```
# my_modules/hello/BUILD.bazel
load("//build/kernel/kleaf:kernel.bzl", "ddk_module")

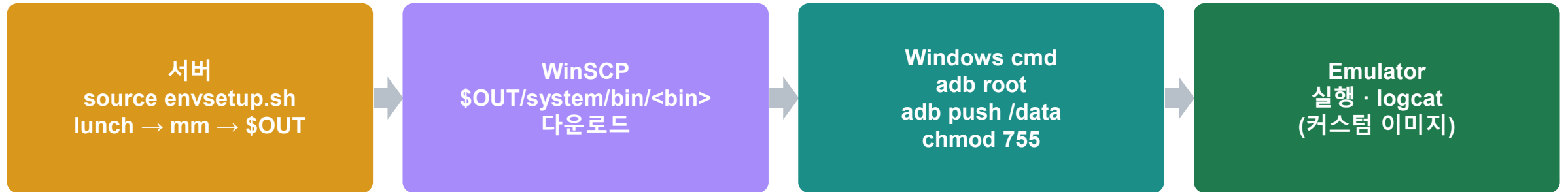
ddk_module(
    name = "hello_android",
    srcs = ["hello_android.c"],
    out = "hello_android.ko",
    kernel_build = "//common-modules/virtual-device:virtual_device_x86_64",
)

$ tools/bazel build //my_modules/hello:hello_android --check_visibility=false
```

- kernel_module() 은 Android 16 에서 deprecated → ddk_module()
- Kbuild / Makefile 을 만들지 않는다 — 자동 생성, 있으면 충돌
- vermagic 이 Emulator 커널과 같아야 insmod 성공 (같은 소스에서 빌드)
- GKI 6.12 커널 → prebuilts/qemu-kernel 교체 → m emu_img_zip → 커스텀 Emulator



Day 2~4 공통 배포 절차 — 오늘 만든 파이프라인



- lunch 는 터미널 세션마다 다시 실행
- Native 바이너리·라이브러리 변경 → mm → adb push (Emulator 재시작 불필요)
- 커널 · APEX · RRO 등 시스템 이미지 변경 → m emu_img_zip → WinSCP → Emulator 재시작
- Soong 분석은 RAM 20 GB+ — 동시 빌드는 조 단위로, swap 32 GB 권장

▶ [애니메이션 8 · AOSP 빌드·배포 파이프라인](#)



Day 1 Worksheet — 20문제, 20분

유형	문제	범위
객관식	Q1 ~ Q8	init · Binder 목적 · oneway · TransactionTooLarge · Messenger · adb root · asInterface · ddk_module
단답형	Q9 ~ Q13	--start-system-server · envsetup.sh · mm · Binder Thread · Zygote
빈칸 채우기	Q14 ~ Q17	Stub/Proxy · 1 MB/PFD · android:process/linkToDeath · product/userdebug
순번 지정	Q18 ~ Q19	Boot Sequence · Native 배포 절차
선잇기	Q20	init · Zygote · system_server · ServiceManager · Binder Driver ↔ 역할

- Day1_Worksheet.html 을 열어 풀고 채점하기 → 문제별 해설 확인
- 종이로 풀 경우 인쇄 (문제만) → 강사가 정답·해설 모두 보기로 풀이
- 16 / 20 이상이면 오늘 내용을 충분히 이해한 것



오늘 확인한 것

Boot

init(1) → Zygote → system_server. 모든 App 의 PPID = Zygote

Binder

Proxy → Driver(1-copy) → Binder Thread → Stub. 호출자 UID 는 커널이 보

AIDL

동기는 블로킹, oneway + Callback 은 비동기. 콜백·binderDied 는 Binder Thread

한계

Transaction Buffer 1 MB 공유 → 대용량은 FD(Pipe/ashmem)

파이프라인

lunch → mm → WinSCP → adb push → 실행. Day 2~4 공통

